

Operating Systems and System Programming

Practical – 4

Task-1:

Aim

To create multiple child processes and synchronize their completion using `wait()` and `waitpid()`, and to create, observe, and eliminate zombie processes using proper synchronization techniques.

Algorithm

1. Start the program.
2. Create multiple child processes using `fork()`.
3. Display the PID of each child process.
4. Use `wait()` to wait for any child process to complete.
5. Use `waitpid()` to wait for a specific child process.
6. Create a zombie process by allowing the child to terminate while the parent does not call `wait()`.
7. Use `ps` to observe the zombie process.
8. Modify the program to use `wait()` and collect the terminated child.
9. Verify that the zombie process is eliminated.
10. Stop the program.

Step-1:

```
allur@AKSHAYA:~$ nano practical-4.c
```

Step-2:

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 practical-4.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    int i;  
    pid_t pid[3];  
  
    for (i = 0; i < 3; i++)  
    {  
        pid[i] = fork();  
  
        if (pid[i] == 0)  
        {  
            printf("Child %d: PID = %d\n", i + 1, getpid());  
            sleep(i + 1);  
            return 0;  
        }  
    }  
  
    wait(NULL);  
    printf("Parent: One child completed using wait()\n");  
  
    waitpid(pid[2], NULL, 0);  
    printf("Parent: Specific child completed using waitpid()\n");  
  
    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute  
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
        waitpid(pid[2], NULL, 0);  
        printf("Parent: Specific child completed using waitpid()\n");  
  
        wait(NULL);  
  
        return 0;  
    }  
}
```

Step-3:

```
allur@AKSHAYA:~$ gcc practical-4.c  
allur@AKSHAYA:~$ ./a.out  
Child 1: PID = 655  
Child 2: PID = 656  
Child 3: PID = 657  
Parent: One child completed using wait()  
Parent: Specific child completed using waitpid()  
allur@AKSHAYA:~$
```

Task-2:

Step-1: Program to Create a Zombie

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 practical4_task2.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child terminating...\n");  
        return 0;  
    }  
    else  
    {  
        printf("Parent PID = %d\n", getpid());  
        printf("Child PID = %d\n", pid);  
  
        sleep(30);  
    }  
  
    return 0;  
}
```

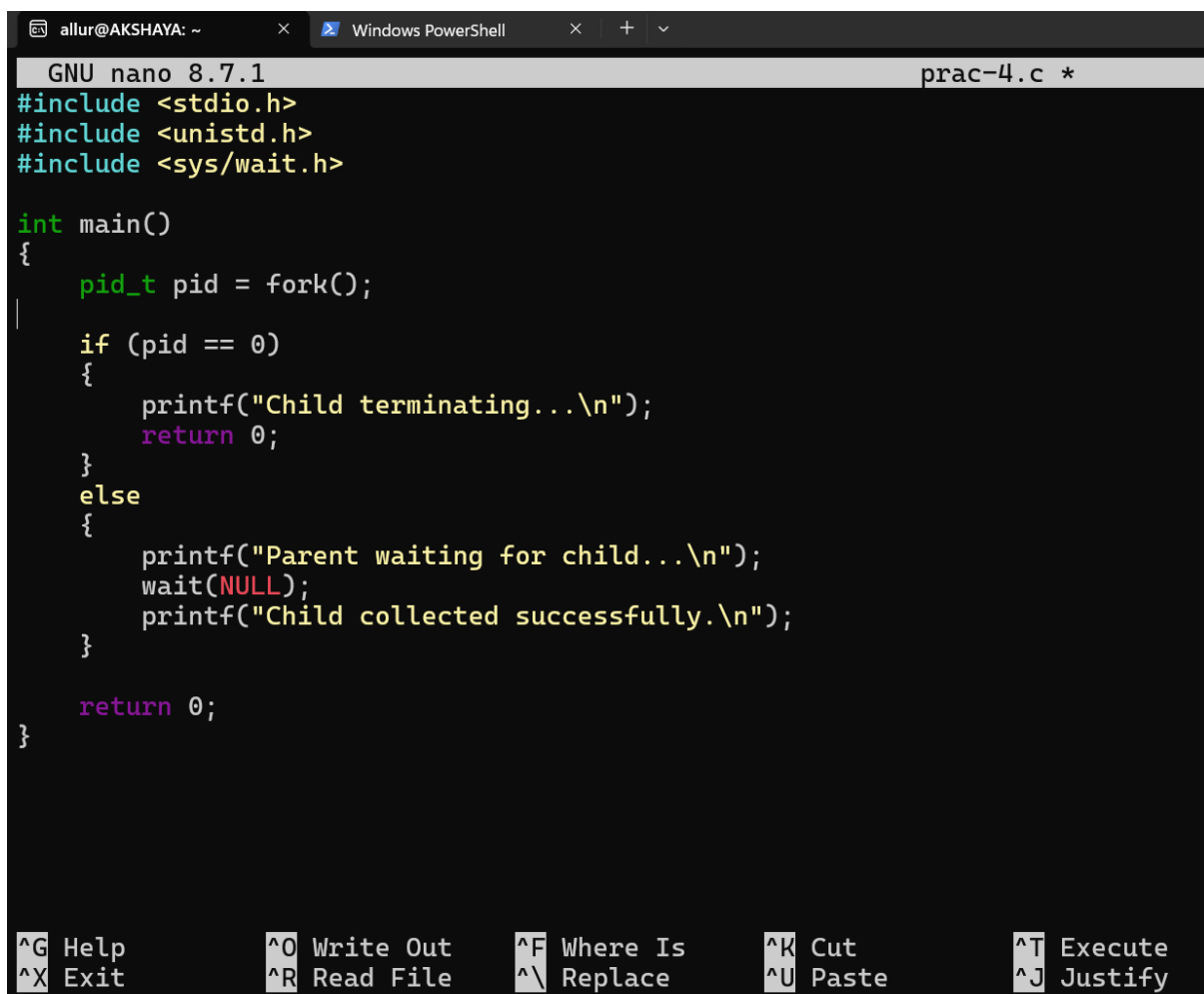
Help Write Out Where Is Cut Execute
Exit Read File Replace Paste Justify

Step-2: Compile and Run

```
allur@AKSHAYA:~$ gcc practical4_task2.c  
allur@AKSHAYA:~$ ./a.out  
Parent PID = 1020  
Child PID = 1021  
Child terminating...  
allur@AKSHAYA:~$ |
```

```
Child terminating...  
allur@AKSHAYA:~$ ps -o pid,ppid,state,cmd  
PID PPID S CMD  
406 405 S -bash  
1068 406 R ps -o pid,ppid,state,cmd  
allur@AKSHAYA:~$ |
```

Modified Program to Prevent Zombie



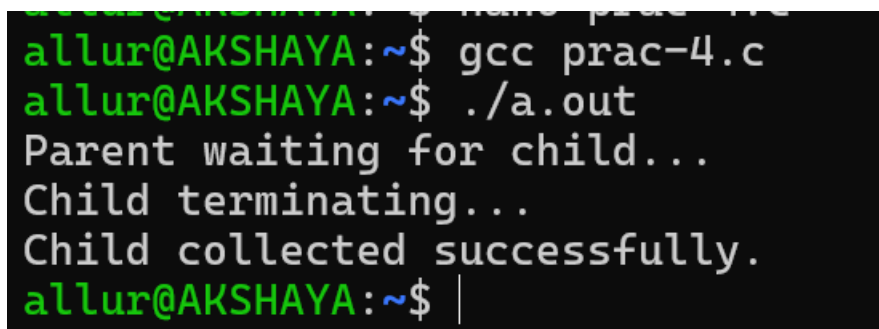
```
GNU nano 8.7.1 prac-4.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child terminating...\n");
        return 0;
    }
    else
    {
        printf("Parent waiting for child...\n");
        wait(NULL);
        printf("Child collected successfully.\n");
    }

    return 0;
}
```

Step-3:



```
allur@AKSHAYA:~$ gcc prac-4.c
allur@AKSHAYA:~$ ./a.out
Parent waiting for child...
Child terminating...
Child collected successfully.
allur@AKSHAYA:~$ |
```

Result

Multiple child processes were successfully created and synchronized using `wait()` and `waitpid()`. A zombie process was successfully observed using the process table and was eliminated by using proper `wait()` synchronization.